



Experiment 03: Continuous Integration Pipeline using Jenkins / GitHub Actions

Experiment No.	03
Mapped CO	CO3
Aim	To set up a Continuous Integration (CI) pipeline that automatically builds and tests an application on every code change using GitHub Actions, with Jenkins as an alternative.
Tools / Software	GitHub account; Jenkins (optional); A sample application (Python or Node.js); Web browser / terminal

Theory

Continuous Integration (CI) is the practice of frequently merging developer changes into a shared mainline repository. Each change is automatically verified through a build and test process so that integration errors and defects are detected early. A CI server or cloud CI service checks out the latest source code, installs dependencies, builds the application and runs test cases. The final result is shown as pass or fail feedback to the development team.

GitHub Actions implements CI as code using YAML workflow files stored in the `.github/workflows` directory of the repository. A workflow is triggered by events such as push or pull_request and is made up of jobs and steps that run on a hosted runner. Jenkins can perform the same task through a Pipeline job using a Jenkinsfile and webhooks from GitHub.

Procedure

1. Create or open a GitHub repository containing a sample Python or Node.js application with a test suite.
2. Create the folder `.github/workflows` in the repository and add a workflow file named `ci.yml`.
3. Define workflow triggers for push and pull_request events on the main branch.
4. Add workflow steps to check out the code, set up the runtime, install dependencies and execute tests.
5. Commit and push the workflow file to GitHub.
6. Open the Actions tab in GitHub and observe the workflow run. Confirm that the build and test jobs complete successfully with a green check.
7. As an alternative, create a Jenkins Pipeline job, point it to the repository, configure the GitHub webhook, and use a Jenkinsfile containing build and test stages.

Sample Code / Commands

`.github/workflows/ci.yml` (Python example)

```
name: CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
```



```
build-test:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with:
        python-version: '3.12'
    - run: pip install -r requirements.txt
    - run: pytest -q
```

Jenkinsfile (declarative pipeline alternative)

```
pipeline
{
  agent any
  stages {
    stage('Build') {
      steps { sh 'pip install -r requirements.txt' }
    }
    stage('Test') {
      steps { sh 'pytest -q' }
    }
  }
}
```

Observation / Experiment Output

Instructions to students: Paste screenshots or copied outputs in the boxes below after completing CI workflow setup and execution. Keep screenshots clear and readable.

Name	Roll No.	Batch	Date
Vighnesh Pingale	48	B2	11/8/26

1. GitHub Repository and Sample Application Output

The screenshot shows the GitHub repository page for 'ci-demo-app'. It includes a 'Quick setup' section with instructions for cloning the repository using HTTPS or SSH. Below this, there is a section titled '...or create a new repository on the command line' which provides a series of git commands to initialize a new repository, add a README file, commit the changes, and push them to the main branch.

```
echo "# ci-demo-app" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/Vighnesh279/ci-demo-app.git
git push -u origin main
```



PARSHWANATH CHARITABLE TRUST'S
A.P. SHAH INSTITUTE OF TECHNOLOGY
Department of Computer Science and Engineering
Data Science



Paste output / screenshot here

2. CI Workflow File / Jenkinsfile Output

```
1 name: Python CI
2
3 on:
4   push:
5     branches: [main]
6   pull_request:
7     branches: [main]
8
9 jobs:
10  test:
11    runs-on: windows-latest
12
13    steps:
14      - name: Checkout code
15        uses: actions/checkout@v4
16
17      - name: Set up Python
18        uses: actions/setup-python@v5
19        with:
20          python-version: "3.11"
21
22      - name: Install Dependencies
23        run: |
24          python -m pip install --upgrade pip
25          pip install -r requirements.txt
```

3. Pipeline Trigger Output

Paste screenshot/output showing push or pull request event triggering the GitHub Actions workflow or Jenkins build.

CI

✓ Add CI pipeline #1

Summary

All jobs

build-test

Run details

Usage

Workflow file

Triggered via push 9 minutes ago

Status: Success

Total duration: 24s

Artifacts: -

ci.yml

on: push

build-test 13s

Annotations

1 warning

build-test

Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-python@v5. F...

Show more



PARSHWANATH CHARITABLE TRUST'S
A.P. SHAH INSTITUTE OF TECHNOLOGY
Department of Computer Science and Engineering
Data Science



4. Build and Test Execution Output

The screenshot shows the GitHub Actions interface for a workflow named 'build-test'. The workflow is in a 'succeeded' state, having completed 9 minutes ago. The left sidebar shows the workflow file and a list of jobs, with 'build-test' selected. The main area displays the job's annotations, including a warning and a list of steps with their durations.

Step	Duration
Set up job	2s
Run actions/checkout@v4	1s
Run actions/setup-python@v5	0s
Run pip install -r requirements.txt	6s
Run pytest -q	1s
Post Run actions/setup-python@v5	0s
Post Run actions/checkout@v4	0s
Complete job	0s

5. Successful CI Status Output

The screenshot shows the GitHub Actions interface for a repository named 'Vighnesh279 / ci-demo-app'. The 'Actions' tab is selected, showing a list of workflow runs. The left sidebar shows the workflow file and a list of jobs, with 'Python CI' selected. The main area displays the workflow runs, including the 'Create ci.yml' and 'Rename req.text to requirements.txt' jobs.

Workflow	Event	Status	Branch	Actor
Create ci.yml	Push	Succeeded	main	Vighnesh279
Rename req.text to requirements.txt	Push	Succeeded	main	Vighnesh279



PARSHWANATH CHARITABLE TRUST'S
A.P. SHAH INSTITUTE OF TECHNOLOGY
Department of Computer Science and Engineering
Data Science



GitHub Repository Link:

<https://github.com/Vighnesh279/ci-demo-app>

GitHub Actions / Jenkins Build Link:

<https://github.com/Vighnesh279/ci-demo-app/actions>

Final Observation Statement:

The Continuous Integration (CI) pipeline was successfully configured using GitHub Actions. The workflow automatically built the application and executed the test cases whenever code was pushed to the repository. This provided quick feedback on the build and test status, improving code quality and reliability.

Expected Output

On every push or pull request, the CI workflow should run automatically, install dependencies, execute the test suite and report a pass/fail status in the GitHub Actions tab or Jenkins build history.

Conclusion

An automated Continuous Integration pipeline was configured so that each committed code change triggers a build and test cycle, providing rapid feedback on code quality and application health.

Viva / Review Questions

1. What problem does Continuous Integration solve?
2. What is the role of a webhook in a CI pipeline?
3. Differentiate Continuous Integration, Continuous Delivery and Continuous Deployment.
4. What is a runner in GitHub Actions?

Marks: ____ / 10

Date: _____ **Signature of Faculty:** _____